ITELEC 4100 - Advanced Web Development
1st SEMESTER A.Y. 2026-2027
FINAL TERM PROJECT

NAMES:	SCORE:
	DATE:
	SECTION:

Overview: This assessment serves as the capstone for the term, requiring the synthesis of Activity 5 (Laravel MVC Foundation), Activity 6 (React Integration & Permissions), and the Final Project requirements into a singular, production-grade application: FinalTermLaravel.

Students will transition from a traditional server-side rendered approach to a modern, decoupled architecture. The objective is to build a robust backend using Laravel that serves a dynamic, component-based frontend powered by React and Inertia.js.

Objectives:

- **MVC Architecture:** Implement an organized structure that cleanly separates business logic, data persistence, and user presentation.
- **Component-Based UI:** Design reusable UI components using React, leveraging JSX syntax and the Virtual DOM for efficient rendering.
- **Advanced Logic & Security:** Deploy secure backend logic including middleware, Eloquent ORM relationships, and authentication, and role-Based Access Control (RBAC).
- **Rapid & Elegant Development:** Adhere to Laravel's philosophy of clean, expressive syntax to build scalable APIs and interactive interfaces.

Activity 5: Backend Infrastructure

• **Database Connection & Migrations:** Configure database credentials in .env. Create normalized migration tables (e.g., product_categories, products) with primary keys, foreign key constraints (cascadeOnDelete()), and timestamp fields.

• **Eloquent Models & Relationships:** Define mass-assignment protection using $fillable arrays across all models. Declare explicit Eloquent relationship methods such as hasMany and belongsTo.

• **Resource Controllers:** Generate RESTful controllers with standard CRUD action methods (index, create, store, show, edit, update, destroy) and strict server-side input validation via $request->validate().

• **Modular Routing:** Organize routes into dedicated files under routes/ (e.g., product-categories.php, products.php) using named routes, prefixes, and route groups

Activity 6: Modern Frontend Integration (React & Inertia.js)

• **Inertia & Package Setup:** Install backend adapter (composer require inertiajs/inertia-laravel) and frontend packages (react,
react-dom, @inertiajs/react, @vitejs/plugin-react).

• **Middleware & Root Blade:** Publish and register HandleInertiaRequests middleware in bootstrap/app.php. Create resources/views/app.blade.php containing @viteReactRefresh, @vite(...), @inertiaHead, and @inertia.

• **Vite & Entry Point:** Configure vite.config.js with the React plugin. Set up resources/js/app.jsx utilizing createInertiaApp and
resolvePageComponent.
• **Controller Rendering:** Refactor resource controllers to return dynamic React page components via Inertia::render('PageName', $data).

Final Project: Production Grade Web Application with Headless Auth & Access Control (Fortify + Spatie)
• **Laravel Fortify Authentication:** Install Fortify for headless authentication (registration, login, password reset). Direct authentication endpoints to render custom React Inertia views.
• **Spatie Roles & Permissions:** Install spatie/laravel-permission. Run migration scripts for roles, permissions, and pivot tables. Attach HasRoles trait to the User model.
• **Database Seeding:** Create a RolesAndPermissionsSeeder to seed default roles (e.g., Admin, Manager, User) and assign granular permissions.
• **Authorization Guards:** Enforce protection using Laravel middleware (auth, role:Admin) on backend routes and conditionally render React UI elements based on authenticated user roles.

Project Rubrics - Scoring

Project and Presentation		**Scores**		
Category	**Specification**	**Excellent**	**Good**	**Poor**
Architecture (30%)	Correct MVC flow; proper migration usage; Eloquent ORM implementation.	**30**	**20**	**10**
Frontend & Integration (30%)	Conversion of Blade to React; efficient use of JSX and Virtual DOM; Inertia.js connectivity.	**30**	**20**	**10**
Security and Permissions (25%)	Correct Spatie RBAC implementation; `@csrf` protection; conditional React rendering based on roles.	**25**	**15**	**5**
Code Quality (15%)	Adherence to clean, expressive syntax; organized file structure; proper naming conventions.	**15**	**10**	**5**
Maximum Score		**100**	**65**	**30**
Total				